

Conceptos intermedios

30. Programación orientada a objetos

- **Clases** : Plantilla o molde lógico que define la estructura y el comportamiento de un tipo de dato personalizado, utiliza la palabra clave “class” para agrupar variables y funciones relacionadas bajo un mismo espacio de nombres, estableciendo un "plano" que el motor de PHP utiliza para crear entidades concretas en memoria. Las clases permiten implementar la abstracción y el encapsulamiento, organizando el código en unidades modulares e independientes y se utilizan como la base del diseño de software moderno para representar entidades del dominio (como un Usuario, una ConexionBD o un Producto), facilitando el mantenimiento y la escalabilidad del sistema al centralizar la lógica en componentes reutilizables.
- **Objetos** : Instancia concreta y específica de una clase, creada en tiempo de ejecución mediante la palabra clave “new”, mientras que la clase es el diseño teórico, el objeto es la entidad viva que ocupa un espacio en la memoria RAM y posee valores particulares, PHP maneja los objetos mediante referencias, lo que significa que al asignar un objeto a una nueva variable, ambas apuntan a la misma dirección de memoria.
Se utilizan para manipular datos reales siguiendo las reglas definidas en su clase madre, permitiendo que múltiples instancias de una misma categoría (por ejemplo, dos objetos Usuario con nombres distintos) coexistan y operen de forma independiente dentro del mismo script.
- **Propiedades** : Variables declaradas dentro de una clase que representan el estado o las características de los objetos resultantes, en PHP las propiedades deben definirse con un modificador de visibilidad (public, protected o private) y, desde versiones recientes, pueden incluir una declaración de tipo (por ejemplo public string \$nombre;), estas variables almacenan la información que diferencia a un objeto de otro.
Se utilizan para persistir datos internamente durante el ciclo de vida del objeto, actuando como el contenedor de información que los métodos de la clase consultarán o modificarán para realizar sus tareas lógicas.
- **Métodos** : Funciones definidas dentro del cuerpo de una clase que representan las acciones o comportamientos que un objeto puede realizar, al igual que las propiedades, los métodos utilizan modificadores de acceso para controlar quién puede invocarlos y a través de los métodos, los objetos interactúan con sus propias propiedades o con otros objetos, procesando información y devolviendo resultados. Se utilizan para encapsular la lógica operativa (por ejemplo guardarEnBaseDeDatos(), calcularTotal() o enviarEmail()), asegurando que el estado interno del objeto solo sea modificado mediante reglas controladas y validadas, evitando la manipulación externa impredecible de los datos.
- **\$this** : (Utilizada como \$this dentro del código) Referencia especial que apunta exclusivamente a la instancia actual del objeto desde el cual se está ejecutando un método, siendo el mecanismo que permite a un objeto acceder a sus propias

propiedades y métodos internos desde dentro de la clase, sin el uso de esta pseudo-variable, el motor de PHP no podría distinguir entre una variable local de la función y un atributo perteneciente al objeto.

Se utiliza sistemáticamente para la lectura y escritura de propiedades internas (por ejemplo `$this->id = 5;`) y para invocar otros métodos de la misma instancia, garantizando la cohesión interna y la comunicación entre los componentes del objeto.

31. Tipos de encapsulación

- **public** : Establece el nivel de visibilidad más permisivo en PHP, indicando que la propiedad o el método marcado puede ser accedido y manipulado desde cualquier lugar del programa, esto incluye el acceso desde el interior de la propia clase, desde clases que heredan de ella y, lo más importante, desde instancias externas (fuera del cuerpo de la clase), si no se especifica explícitamente un modificador al declarar un método, PHP lo asume como público por defecto.

Se utiliza para definir la "interfaz pública" del objeto, es decir, los puntos de entrada y acciones que el resto de la aplicación necesita conocer para interactuar con la entidad, como métodos de procesamiento de datos o constantes de configuración general.

- **private** : Restringe el acceso a una propiedad o método exclusivamente al interior de la clase donde fue definido, ni las instancias externas ni las clases que heredan (subclases) pueden leer o modificar los elementos marcados como privados.

Este nivel de visibilidad es el pilar del encapsulamiento estricto, ya que protege el estado interno del objeto de interferencias accidentales o malintencionadas desde el exterior y se utiliza para ocultar detalles de implementación complejos, almacenar datos sensibles que solo deben ser procesados por la lógica interna de la clase y garantizar que el estado del objeto solo cambie a través de métodos públicos controlados que validen la información antes de guardarla.

- **protected** : Permite que las propiedades o métodos sean accesibles únicamente desde el interior de la propia clase y desde cualquier clase que herede de ella (hijas), impidiendo el acceso desde instancias externas u otros componentes ajenos a la jerarquía de herencia.

Se utiliza fundamentalmente en arquitecturas basadas en la extensión de clases, permitiendo que las clases derivadas reutilicen y compartan lógica o datos de la clase base sin exponer esa información al resto de la aplicación, facilitando la especialización de comportamientos mientras se mantiene un nivel de seguridad y orden en la estructura de objetos.

32. Herencia

- **extends** : Crea una relación jerárquica donde una clase (hija o subclase) adquiere automáticamente todas las propiedades y métodos no privados de otra clase (padre o superclase).

Al utilizar esta instrucción, el motor de PHP permite que el código sea altamente reutilizable, evitando la duplicación de lógica común y estableciendo un contrato de tipo donde la clase hija "es un" tipo de la clase padre y se utiliza para construir arquitecturas modulares donde se definen comportamientos generales en una base

(como una clase Vehiculo) y comportamientos específicos en sus derivados (como Coche o Moto).

- **Sobrescritura de métodos** : Capacidad de una subclase para proporcionar una implementación específica de un método que ya ha sido definido en su clase padre, para que esto ocurra la sobrescritura, la clase hija debe declarar un método con el mismo nombre y, preferiblemente, la misma firma (parámetros y tipos) que el original.

El motor de PHP ejecutará la versión del método perteneciente al objeto instanciado, permitiendo que la subclase modifique o refine el comportamiento heredado sin alterar la clase base y se utiliza para adaptar funcionalidades genéricas a necesidades particulares, como por ejemplo cambiar la lógica de un método `calcularImpuesto()` en diferentes categorías de productos, manteniendo la coherencia de la interfaz pero variando el resultado operativo.

33. Polimorfismo

- **¿Qué es?** : Principio de la programación orientada a objetos que permite que diferentes clases respondan al mismo mensaje (o llamado a un método) de maneras distintas, en PHP esto se logra principalmente a través de la herencia y las interfaces, permitiendo que un programa trabaje con objetos de diversas clases como si fueran instancias de una misma familia o contrato, el motor de PHP resuelve en tiempo de ejecución qué implementación específica del método debe ejecutarse basándose en la naturaleza del objeto real que se está manipulando.

Se utiliza para escribir código genérico y flexible que puede procesar colecciones de objetos heterogéneos (por ejemplo una lista de diferentes tipos de Pagos) invocando los mismos métodos (como `procesar()`) sin necesidad de conocer los detalles internos de cada uno.

- **Métodos sobrescritos** : Permite el polimorfismo basado en la herencia, que cuando una subclase redefine un método que ya existe en su clase padre, está proporcionando una especialización de ese comportamiento, se manifiesta cuando el código interactúa con una variable de tipo "clase padre", pero el motor de PHP ejecuta la versión del método de la "clase hija" que realmente contiene la variable. Se utiliza para asegurar que, aunque el sistema trate a todos los objetos bajo una categoría general, cada uno mantenga su comportamiento específico y correcto (por ejemplo, que un método `dibujar()` funcione de forma distinta para un objeto `Circulo` que para un `Cuadrado`).
- **Interfaces** : Estructura definida con la palabra clave "interface" que declara un conjunto de métodos que una clase debe implementar, pero sin proporcionar el código de los mismos, a diferencia de la herencia de clases, una clase en PHP puede implementar múltiples interfaces simultáneamente. Las interfaces representan el nivel más puro de polimorfismo, ya que definen "qué" puede hacer un objeto (un contrato) sin importar "cómo" lo hace ni de qué familia de clases proviene y se utilizan para desacoplar la lógica de la aplicación de las implementaciones concretas, permitiendo intercambiar componentes (por ejemplo cambiar un motor de base de datos MySQL por uno PostgreSQL) siempre que ambos respeten los métodos definidos en la interfaz común.

34. Clases abstractas

- **abstract classes** : Clase que no puede ser instanciada directamente (no se puede crear un objeto con new) y actúa exclusivamente como una base conceptual para otras clases, definiéndose por utilizar la palabra clave abstract y tiene como objetivo principal proporcionar una estructura común, permitiendo definir tanto métodos con implementación completa como métodos que carecen de ella, a diferencia de las interfaces, las clases abstractas pueden contener propiedades (variables de estado) y métodos con lógica compartida que todas las subclases heredarán.
Se utilizan para modelar identidades generales que no tienen sentido por sí solas en el dominio del negocio (por ejemplo una clase MetodoPago), obligando a que se creen versiones concretas y específicas (como TarjetaCredito o PayPal) para poder operar.
- **abstract methods** : Declaración de función dentro de una clase abstracta que define la firma del método (nombre, parámetros y tipo de retorno) pero no contiene cuerpo ni lógica, al marcar un método como abstract, el motor de PHP impone una restricción estricta: cualquier clase hija que herede de la clase abstracta está obligada a proporcionar una implementación real y completa de dicho método, si la subclase no define el método, PHP lanzará un error fatal en tiempo de compilación. Se utilizan para garantizar que todas las clases derivadas compartan una interfaz común y realicen acciones críticas de manera obligatoria, pero permitiendo que cada una decida internamente cómo llevar a cabo esa acción (por ejemplo, obligar a que todos los tipos de Archivo tengan un método leer(), pero con lógicas distintas para un CSV o un JSON).

35. Métodos y propiedades estáticas

- **static** : Declara propiedades y métodos que pertenecen a la estructura de la clase y no a las instancias u objetos individuales derivados de ella, al marcar un elemento como estático, el motor de PHP reserva un único espacio en memoria para ese dato durante toda la ejecución del script, independientemente de cuántas veces se instancie la clase.
Los métodos estáticos se invocan directamente a través del nombre de la clase utilizando el operador de resolución de ámbito (::) y no tienen acceso a la pseudo-variable \$this, ya que no operan sobre el estado de un objeto particular y se utilizan para implementar funciones de utilidad global, contadores de instancias, constantes de configuración compartidas o para aplicar patrones de diseño que requieren un punto de acceso único y centralizado.
- **self** : Conector lógico utilizado dentro de los métodos de una clase para hacer referencia a la propia definición de la clase actual, a diferencia de las referencias a instancias, self se emplea junto al operador de resolución de ámbito (::) para acceder exclusivamente a miembros estáticos y constantes de clase desde su interior.
Es una referencia estática que se resuelve en tiempo de compilación basándose en la clase donde se encuentra escrito el código, garantizando que el acceso se mantenga vinculado a la clase original y se utiliza para invocar métodos estáticos internos, recuperar valores de propiedades estáticas compartidas por todos los objetos de esa categoría y gestionar constantes de clase, asegurando la cohesión del código sin depender de nombres de clase externos.

36. Constantes de clase

- **¿Qué son?** : Valores fijos que pertenecen a la definición de una clase y que no pueden ser modificados una vez que el script ha sido compilado, se declaran mediante la palabra clave `const` y, por convención, sus identificadores se escriben íntegramente en mayúsculas y a diferencia de las propiedades estándar, las constantes de clase no requieren un modificador de visibilidad para ser accedidas (aunque desde PHP 7.1 pueden ser `public`, `private` o `protected`) y se accede a ellas mediante el operador de resolución de ámbito (`::`) sin necesidad de instanciar un objeto.

Se utilizan para almacenar datos de referencia permanentes que son intrínsecos a la naturaleza de la clase, como códigos de error, configuraciones de estado fijas o factores de conversión matemáticos.

- **Traits** : Agrupa métodos que pueden ser insertados en múltiples clases de forma independiente a la jerarquía de herencia, en PHP se utiliza la palabra clave `trait` para definir estos conjuntos de funciones y la palabra clave `use` dentro de una clase para incorporarlos, al no ser una clase, un `trait` no puede ser instanciado por sí solo, sino que su código se integra virtualmente dentro de la clase que lo utiliza en tiempo de ejecución.

Se utiliza para dotar a clases de diferentes familias de comportamientos comunes, permitiendo que compartan lógica de validación, registro de logs o manipulación de datos sin forzar una relación de parentesco artificial entre ellas.

|
| — **Reutilización de código** : Estrategia de composición horizontal que permite a los desarrolladores evitar la duplicación de métodos en aplicaciones complejas, permitiendo "inyectar" funcionalidades específicas en diversas clases sin las limitaciones de la herencia simple (donde una clase solo puede tener un padre) y al centralizar una función lógica dentro de un `trait`, cualquier cambio realizado en dicha función se refleja automáticamente en todas las clases que la utilizan. Se utiliza para mantener el principio de responsabilidad única y facilitar el mantenimiento del software, permitiendo que componentes especializados de lógica (como un generador de Slugs o un sistema de auditoría) se apliquen de manera transversal en todo el sistema.

- **Características modernas**

|
| — **constructor property promotion** : Introducido en PHP 8.0, es una sintaxis simplificada que permite declarar y asignar propiedades de clase directamente dentro de la firma del método `__construct`, que al añadir un modificador de visibilidad (como `public`, `private` o `protected`) a los parámetros del constructor, el motor de PHP realiza automáticamente tres acciones: declara la propiedad en la clase, define su tipo y le asigna el valor recibido en el argumento al instanciar el objeto.
| Esta característica elimina la redundancia de escribir el nombre de la variable en cuatro lugares distintos (declaración, parámetro, asignación y cuerpo) y se utiliza para reducir drásticamente la verbosidad del código (boilerplate), especialmente en clases de transferencia de datos (DTOs) y servicios con múltiples dependencias inyectadas.

— **readonly properties** : Incorporados en PHP 8.1, son atributos de clase que solo pueden ser asignados una única vez, generalmente durante la inicialización en el constructor, y una vez que la propiedad ha recibido un valor, el motor de PHP bloquea cualquier intento posterior de modificación, lanzando un error fatal si se intenta sobrescribir desde dentro o fuera de la clase, se definen por instancia y pueden almacenar cualquier tipo de dato dinámico. Se utilizan para garantizar la inmutabilidad de los objetos después de su creación, protegiendo el estado crítico de entidades de negocio y asegurando que los datos de configuración o identificadores permanezcan constantes durante todo el ciclo de vida del objeto en memoria.

— **enums (PHP 8.1)** : Introducidos en PHP 8.1, son un tipo de dato especial que permite definir una lista cerrada y estricta de valores posibles para una variable, actuando como objetos que pueden contener métodos y ser utilizados en declaraciones de tipo (type hinting), garantizando que una función solo acepte uno de los casos definidos (por ejemplo, `Status::Active` o `Status::Pending`).

Existen en dos variantes: Pure Enums (simples etiquetas) y Backed Enums (asociados a un valor escalar como string o int) y son utilizados para modelar conjuntos de opciones fijas y mutuamente excluyentes, eliminando errores de lógica derivados del uso de cadenas de texto arbitrarias y mejorando la legibilidad y seguridad de la arquitectura del software.

37. Namespaces

- **namespaces** : Contenedor lógico definido mediante la palabra clave namespace al inicio de un archivo PHP, diseñado para agrupar clases, funciones y constantes bajo un identificador único, siendo su función principal resolver el conflicto de nombres (name collision) que ocurre cuando dos librerías diferentes intentan declarar una clase con el mismo nombre, lo que hace es que el motor de PHP organiza estas estructuras de forma jerárquica, similar a una estructura de directorios en un sistema de archivos, permitiendo que coexistan múltiples entidades homónimas siempre que pertenezcan a espacios distintos.

Se utiliza para encapsular componentes de software, facilitar la integración de paquetes de terceros (vía Composer) y establecer una estructura clara y profesional en el código fuente de aplicaciones de gran escala.

- **use** : Instrucción de importación que permite al desarrollador emplear clases, interfaces o rasgos de otros espacios de nombres sin necesidad de escribir su ruta completa (Fully Qualified Name) en cada mención, al declarar un “use” al principio del archivo, se crea un alias interno en el contexto actual del script y opcionalmente, se puede emplear la cláusula “as” para renombrar la entidad importada y evitar conflictos adicionales.

Se utiliza para mejorar drásticamente la legibilidad del código, permitiendo el uso de nombres cortos y claros mientras se mantiene la separación lógica de las librerías externas o módulos internos.

- **Organización modular** : Estrategia de diseño que utiliza los espacios de nombres para segmentar una aplicación en componentes o módulos independientes y reutilizables, en esta arquitectura cada módulo (como Auth, Billing o Api) reside en su propio namespace, siguiendo generalmente el estándar PSR-4, que vincula la jerarquía de los espacios de nombres directamente con la estructura de carpetas del servidor.

Esta disposición permite que los cargadores automáticos (autoloaders) localicen y carguen los archivos necesarios de forma eficiente bajo demanda y se utiliza para reducir el acoplamiento entre las distintas partes del sistema, facilitar el trabajo en equipo sobre diferentes secciones del proyecto y permitir que la aplicación crezca en complejidad sin perder la trazabilidad ni el orden de sus componentes.

38. Funciones avanzadas

- **Funciones anónimas** : También conocidas como lambdas, son funciones que se definen sin un identificador o nombre específico y que se tratan como objetos de la clase interna Closure, estas se asignan directamente a variables o se pasan como argumentos a otras funciones (callbacks) y el motor de PHP las procesa en tiempo de ejecución, permitiendo una programación más flexible y dinámica.
Se utilizan principalmente para tareas de corta duración que no requieren ser reutilizadas globalmente, como transformaciones rápidas de datos en `array_map()` o filtros personalizados en `array_filter()`, manteniendo el espacio de nombres global libre de funciones de un solo uso.
- **closures** : Función anónima con la capacidad de "encapsular" y transportar variables del entorno o ámbito donde fue creada hacia el ámbito donde será ejecutada, internamente, PHP implementa todos los closures como instancias de la clase Closure, permitiendo que la función mantenga acceso a datos específicos incluso después de que el script haya salido del bloque de código original donde se definió la función.
Se utilizan para crear funciones personalizadas basadas en configuraciones dinámicas, inyectar dependencias en procesos asíncronos y desarrollar arquitecturas de software que requieren que la lógica de ejecución lleve consigo su propio contexto de datos.
- **arrow functions** : Sintaxis abreviada para escribir funciones anónimas de una sola expresión utilizando la palabra clave "fn", su característica técnica más relevante es la captura automática por valor de las variables del ámbito padre, eliminando la necesidad de declarar la cláusula use.
Estas funciones siempre devuelven el resultado de la expresión evaluada sin necesidad de la sentencia return y se utilizan para simplificar drásticamente el código cuando se requiere una lógica lineal y concisa, siendo el estándar moderno para operaciones funcionales sobre colecciones de datos donde la brevedad mejora la legibilidad general del script.
- **Paso por referencia** : Mecanismo que permite a una función u operación trabajar directamente sobre la dirección de memoria de una variable original, en lugar de crear una copia de su valor, se activa anteponiendo el símbolo ampersand (&) al parámetro en la definición de la función o en la asignación de la variable, al modificar

un parámetro pasado por referencia, el cambio se refleja instantáneamente en la variable externa al finalizar la ejecución.

Se utiliza para manipular grandes estructuras de datos (como arrays extensos) de forma eficiente sin duplicar el consumo de RAM, o para permitir que una función devuelva múltiples resultados modificando las variables que se le entregan como argumentos.

39. Tipado moderno

- **Tipos en parámetros** : Define qué naturaleza de dato debe recibir una función o método en su firma, al anteponer un tipo (como int, string, array o el nombre de una Class) al nombre de la variable, el motor de PHP valida automáticamente el argumento en el momento de la invocación, si el valor proporcionado no coincide con el tipo declarado y no puede ser convertido mediante coerción, PHP lanzará un error de tipo (TypeError).

Se utiliza para establecer un contrato claro entre el código que llama y la función, eliminando la necesidad de realizar validaciones manuales (is_int, is_string) dentro del cuerpo de la lógica.

- **Tipos de retorno** : Situado al final de la firma de una función (denotada por dos puntos : tipo) que especifica la naturaleza del valor que la función enviará de vuelta al finalizar su ejecución y el motor de PHP supervisa que la sentencia return devuelva exactamente lo prometido en la definición.

Se utiliza para asegurar la consistencia del flujo de datos en aplicaciones complejas, permitiendo que el desarrollador confíe plenamente en el resultado de una función sin temor a recibir valores inesperados que rompan la lógica subsiguiente.

- **union types** : Declara que una propiedad, parámetro o retorno puede aceptar dos o más tipos de datos distintos, separados por el carácter de tubería (|), por ejemplo una declaración int|float indica que el motor aceptará cualquiera de ambos tipos numéricos.

Esta característica técnica resuelve la limitación histórica de tener que elegir un solo tipo o renunciar al tipado por completo y se utilizan para modelar situaciones de negocio comunes donde un dato puede llegar en diferentes formatos (como un ID que puede ser int o string) manteniendo la seguridad del tipado sin recurrir a la ambigüedad total.

- **mixed** : Representa la aceptación de cualquier valor posible dentro de PHP (incluyendo todos los tipos escalares, compuestos y especiales), y aunque técnicamente equivale a no declarar un tipo, el uso de mixed en la firma de una función es una declaración de intención explícita que indica al desarrollador y a las herramientas de análisis estático que la función está diseñada para procesar datos de cualquier naturaleza.

Se utiliza en funciones genéricas de bajo nivel, como formateadores universales o depuradores, donde la lógica interna es capaz de gestionar internamente la diversidad de tipos recibidos.

- **nullable types** : Permite que una declaración de type acepte adicionalmente el valor null, indicándose mediante un signo de interrogación inicial (por ejemplo ?string),

siendo crucial porque, por defecto, un parámetro tipado como string rechazará un valor nulo, el uso de este modificador indica que el dato es opcional o que su ausencia tiene un significado lógico en el sistema.

Se utilizan habitualmente en la recuperación de registros de bases de datos donde un campo puede no tener valor, o en métodos de búsqueda que devuelven un objeto si se encuentra o null si la búsqueda falla.

- **static return type** : Referencia especial que indica que un método debe devolver una instancia de la clase exacta que fue invocada en tiempo de ejecución, y a diferencia del tipo self (que apunta a la clase donde se escribió el código), static respeta la herencia y se ajusta dinámicamente si el método es llamado desde una subclase.

Se utiliza fundamentalmente en el diseño de interfaces fluidas (fluent interfaces) y en patrones de creación (como métodos de factoría), garantizando que el encadenamiento de métodos devuelva siempre el tipo de objeto correcto dentro de una jerarquía de clases.

40. Iteración avanzada

- **foreach por referencia** : Modifica los elementos originales de un array directamente desde el interior del bucle, se activa anteponiendo el símbolo ampersand (&) a la variable de valor en la declaración del bucle (ej. foreach (\$array as &\$valor)), y a diferencia del comportamiento por defecto que trabaja con una copia, esta técnica vincula la variable de iteración a la dirección de memoria exacta de cada elemento del array.

Es una operación de alta eficiencia que evita la duplicidad de datos en memoria al procesar colecciones voluminosas y se utiliza para realizar transformaciones masivas en el lugar (in-place), como normalizar textos, aplicar cálculos de impuestos a precios o actualizar estados de objetos dentro de una lista sin necesidad de reasignar el valor manualmente al índice original, una práctica recomendada es desvincular la referencia con unset(\$valor) al finalizar el bucle para prevenir modificaciones accidentales posteriores.

- **destructuring en foreach** : Extrae y asigna los valores de arrays anidados directamente a variables individuales dentro de la firma del bucle utilizando la sintaxis de corchetes list() o la abreviada [] (por ejemplo foreach (\$usuarios as [\$id, \$nombre])), el motor de PHP descompone cada sub-array interno y mapea sus posiciones a las variables declaradas de forma atómica.

Esta característica técnica elimina la necesidad de acceder a los datos mediante índices numéricos o claves dentro del cuerpo del bucle, reduciendo la verbosidad y el riesgo de errores de índice y se utiliza para procesar de forma elegante resultados de consultas a bases de datos, listas de coordenadas o cualquier estructura de datos matricial donde se conozca de antemano la posición y el significado de los elementos internos, mejorando drásticamente la legibilidad y claridad del código.

41. Iteradores

- **Iterator** : Permite a los objetos ser recorridos en un bucle como si fueran un array, para implementar esta interfaz, una clase debe definir obligatoriamente cinco métodos específicos: current() (valor actual), key() (clave actual), next() (avanzar),

`rewind()` (volver al inicio) y `valid()` (comprobar si hay más datos) y el motor de PHP utilizará estos métodos para controlar el puntero interno del objeto durante una iteración.

Se utiliza para modelar colecciones de datos complejas que no residen íntegramente en la memoria RAM, como grandes conjuntos de resultados de una base de datos, sistemas de archivos extensos o flujos de datos externos que deben ser procesados secuencialmente sin cargar todo el contenido de golpe.

- **iterable** : Actúa como una unión entre los arrays y los objetos que implementan la interfaz Traversable, que cuando se utiliza como declaración de tipo en un parámetro o en un retorno de función, el motor de PHP aceptará cualquier estructura sobre la cual se pueda realizar un bucle `foreach`, siendo el nivel más alto de abstracción para la manipulación de colecciones, permitiendo que una función sea agnóstica respecto a si los datos son una lista simple de valores o un objeto iterador complejo.
Se utiliza para escribir funciones de procesamiento de listas extremadamente flexibles y reutilizables, garantizando que el código funcione correctamente con cualquier estructura de datos que soporte la iteración estándar del lenguaje.
- **generators** : Permite iterar sobre un conjunto de datos sin necesidad de construir un array en memoria, siendo que diferencia de las funciones convencionales que devuelven un único valor y finalizan, un generador utiliza la lógica de interrupción para entregar valores bajo demanda, internamente PHP implementa los generadores mediante la clase `Generator`, la cual es instanciada automáticamente cuando se invoca la función.
Se utilizan para procesar volúmenes masivos de información (como archivos de log de varios gigabytes o millones de registros) con un consumo de memoria constante y mínimo, ya que el motor de PHP solo calcula y mantiene en RAM el elemento que se está procesando en el instante actual de la iteración.
- **yield** : Crea generadores, permitiendo que una función produzca una serie de valores a lo largo del tiempo en lugar de calcularlos todos a la vez, al activarse congela el estado completo de la función incluyendo el valor de sus variables y la posición exacta del código para entregar un dato al llamador y entrar en un estado de pausa, en cuanto se solicita el siguiente valor, la función "despierta" y continúa su ejecución justo donde se detuvo, repitiendo este ciclo hasta que no queden más elementos que procesar.
Esta capacidad la hace extremadamente eficiente para manejar grandes volúmenes de datos, como leer archivos gigantes línea por línea o generar secuencias infinitas, ya que solo consume la memoria necesaria para un solo elemento a la vez, además, PHP ofrece `yield from` para conectar o delegar la producción de valores a otros generadores de forma directa, logrando así estructuras de iteración complejas y personalizadas con un código mucho más limpio, fluido y fácil de mantener que si se definiera manualmente una clase de iterador completa.

42. Manejo de errores real

- **try** : Define un segmento de código que el motor de PHP debe ejecutar bajo una vigilancia especial para detectar cualquier interrupción en el flujo normal y si durante la ejecución de las instrucciones contenidas entre las llaves `{}` se produce un error

crítico o se lanza una excepción, PHP detiene inmediatamente la ejecución de ese bloque y transfiere el control al manejador correspondiente, es importante aclarar que no se puede utilizar de forma aislada, ya que requiere al menos un bloque de captura o un bloque de finalización para ser sintácticamente válido.

Se utiliza para envolver operaciones que dependen de factores externos o inciertos, como conexiones a bases de datos, lectura de archivos del sistema o llamadas a APIs de terceros, asegurando que un fallo en estas tareas no provoque el cierre abrupto del script.

- **catch** : Encargado de interceptar y procesar una excepción o un error que ha sido capturado previamente por un bloque try, este bloque especifica el tipo de objeto que es capaz de manejar (mediante una declaración de tipo en su argumento) y proporciona la lógica necesaria para recuperarse del fallo, registrar el incidente o mostrar un mensaje controlado al usuario.

PHP permite encadenar múltiples bloques catch para gestionar diferentes tipos de errores de forma específica (por ejemplo, uno para errores de base de datos y otro para errores de validación) y se utiliza para implementar la lógica de contingencia, evitar la exposición de trazas de error técnicas al cliente y mantener la estabilidad de la aplicación ante situaciones imprevistas.

- **finally** : Define una serie de instrucciones que el motor de PHP ejecutará siempre, independientemente de si se lanzó una excepción o si el bloque try finalizó con éxito, este bloque se procesa incluso si dentro de los bloques anteriores existe una sentencia return.

Es la última etapa del ciclo de vida de una estructura de control de errores antes de continuar con el resto del script y se utiliza fundamentalmente para tareas de limpieza y liberación de recursos críticos, como cerrar punteros de archivos abiertos, finalizar conexiones a servicios externos o limpiar variables temporales de memoria, garantizando que el sistema regrese a un estado estable sin importar el resultado de la operación principal.

- **throw** : Dispara manualmente una excepción o un error, interrumpiendo el flujo de ejecución actual del programa, ya que al ejecutarse esta instrucción el desarrollador debe proporcionar una instancia de una clase que implemente la interfaz Throwable y una vez lanzada, PHP busca hacia atrás en la pila de llamadas (stack trace) el bloque catch más cercano que pueda manejar ese tipo específico de objeto y si no encuentra ninguno, el script termina con un error fatal.

Se utiliza para validar reglas de negocio complejas, señalar estados inválidos que el programa no puede resolver por sí mismo y forzar al código que invoca una función a manejar situaciones de error predecibles pero críticas.

- **Throwable** : Define los métodos estándar para recuperar información sobre el fallo, como getMessage() (el motivo), getCode() (identificador numérico), getFile() (archivo de origen) y getTraceAsString() (la ruta de ejecución), ninguna clase de usuario puede implementar esta interfaz directamente y en su lugar, deben extender de las clases Exception o Error.

Se utiliza en las firmas de los bloques catch cuando se desea capturar absolutamente cualquier tipo de fallo que pueda ocurrir en el sistema, proporcionando el nivel máximo de abstracción en la gestión de errores.

43. Manejo de datos

- **Formularios**

— **Formularios complejos** : Interfaz de recolección de datos que procesa múltiples tipos de información de forma simultánea, incluyendo arreglos de campos dinámicos, archivos adjuntos y datos anidados, técnicamente PHP gestiona estos formularios interpretando el atributo name de los elementos HTML como estructuras de datos, por ejemplo el uso de corchetes en los nombres de los campos (como productos[]) instruye al motor de PHP para organizar automáticamente las entradas en un array dentro de las superglobales `_POST` o `_GET`, para el manejo de archivos se requiere el atributo `enctype="multipart/form-data"`, lo que activa la población de la superglobal `_FILES`.
Se utilizan para implementar sistemas de pedidos con múltiples artículos, carga masiva de documentos, perfiles de usuario con redes sociales dinámicas y cualquier flujo de trabajo que requiera una transacción de datos heterogénea en una sola petición.

— **Validación de datos** : Verifica que la información recibida por el script cumple con las reglas de negocio, formatos y tipos requeridos antes de permitir su procesamiento o almacenamiento, proporcionando herramientas nativas para este fin, destacando la extensión `filter_var()`, que permite aplicar filtros de validación predefinidos (como `FILTER_VALIDATE_EMAIL`, `FILTER_VALIDATE_INT` o `FILTER_VALIDATE_URL`).
La validación actúa como una barrera lógica que garantiza que el programa no intente operar con datos inconsistentes, malformados o malintencionados y se utiliza sistemáticamente para asegurar que las fechas sean cronológicamente posibles, que las cadenas de texto no excedan los límites de la base de datos y que los campos obligatorios contengan información sustancial, protegiendo la estabilidad del sistema y la calidad de la información persistida.

- **Filtros**

— **filter_var()** : Filtra una variable específica mediante un filtro determinado por el desarrollador, el motor de PHP aplica una lógica interna de validación o saneamiento basada en constantes predefinidas (como `FILTER_VALIDATE_EMAIL` para comprobar formatos de correo o `FILTER_SANITIZE_STRING` para eliminar etiquetas HTML).
Esta función devuelve los datos filtrados en caso de éxito o false si el filtro de validación falla, permitiendo un manejo de errores directo y centralizado y se utiliza habitualmente para procesar variables que ya existen en la memoria del script, como datos que han sido transformados previamente, resultados de funciones de cadena o variables configuradas dinámicamente que requieren cumplir con un formato técnico estricto antes de ser utilizadas en la lógica de

| negocio.

|
|
| **filter_input()** : La función `filter_input()` es un mecanismo de seguridad diseñado para recuperar y filtrar una variable directamente desde una fuente de entrada externa (como `_GET`, `_POST`, `_COOKIE` o `_SERVER`) sin necesidad de acceder manualmente a las variables superglobales, al invocar esta función, se especifica el tipo de entrada, el nombre de la variable y el filtro que se desea aplicar; si la variable no existe en la fuente indicada, la función devuelve `null`.

Esta aproximación técnica es superior al acceso directo mediante arreglos porque mitiga el riesgo de avisos por índices no definidos (Undefined Index) y asegura que los datos externos nunca toquen la lógica del programa sin haber pasado por una capa de validación previa, se utiliza como el estándar profesional para la recepción de parámetros de URL, datos de formularios y cabeceras de servidor, garantizando una superficie de ataque reducida y una mayor integridad de los datos de entrada.

44. Serialización

- **serialize()** : Genera una representación apta para el almacenamiento de cualquier valor que pueda ser manipulado en PHP (excepto el tipo `resource`), esta representación es una cadena de texto con un formato específico que conserva no solo los datos, sino también la estructura y el tipo de la variable original, cuando se aplica a un objeto, PHP intenta ejecutar el método mágico `__sleep()` (si existe) para determinar qué propiedades deben ser serializadas, guardando el nombre de la clase para permitir su posterior reconstrucción.
Se utiliza para guardar estructuras complejas como arrays multidimensionales u objetos en campos de texto de bases de datos, archivos planos o variables de sesión, permitiendo que el estado de la información sobreviva entre diferentes ejecuciones del script.
- **unserialize()** : Transforma una cadena de texto generada previamente por `serialize()` de vuelta a su valor original de PHP, durante este proceso el motor de PHP recrea la estructura de datos, recuperando tipos escalares, arreglos y re-instanciando objetos, si se trata de un objeto PHP intenta invocar el método mágico `__wakeup()` para permitir que el objeto restablezca conexiones o recursos que no fueron serializados (como punteros a bases de datos).
Es una operación crítica de seguridad, ya que no debe utilizarse con datos provenientes de fuentes externas no confiables debido al riesgo de inyección de objetos y se utiliza para recuperar estados guardados, procesar configuraciones serializadas y restaurar el contexto de trabajo de un usuario a partir de datos almacenados en disco o caché.

45. Estado de aplicación

- **Sesiones avanzadas** : Referidas a la configuración y personalización del motor de sesiones de PHP para elevar los estándares de seguridad y escalabilidad más allá del comportamiento por defecto, a través de la función `session_set_cookie_params()` y directivas de `ini_set()`, el desarrollador puede restringir el acceso a la cookie de sesión mediante las banderas `httponly` y `secure`, y definir políticas de `samesite` para

mitigar ataques de falsificación de peticiones en sitios cruzados (CSRF). además, el manejo avanzado incluye la regeneración periódica del identificador de sesión con `session_regenerate_id()` para prevenir el secuestro de sesión (session fixation). Se utiliza en aplicaciones de nivel empresarial para gestionar el estado del usuario en entornos distribuidos, permitiendo que la información de identidad persista de forma cifrada y controlada durante toda la navegación.

- **Gestión de login :** Conjunto de procesos lógicos que validan la identidad de un usuario y establecen un estado de confianza en el servidor, este flujo comienza con la recepción de credenciales a través de una petición POST, continúa con la verificación criptográfica de la contraseña (normalmente mediante `password_verify()`) y finaliza con la escritura de un identificador de usuario en la superglobal `_SESSION`.
Se utiliza como la puerta de enlace de seguridad principal en sistemas que requieren áreas privadas, paneles de administración o perfiles personalizados, asegurando que solo los usuarios legitimados accedan a recursos protegidos.
- **Cookies configuradas :** Pequeños fragmentos de datos enviados por el servidor y almacenados por el navegador que poseen directivas específicas de comportamiento y caducidad, a diferencia de una sesión volátil, una cookie configurada mediante `setcookie()` puede persistir durante meses o años si se le asigna un timestamp futuro, permitiendo funciones como "recordarme en este equipo".
Técnicamente, la configuración incluye el alcance del dominio y la ruta (path), determinando en qué secciones del sitio el navegador debe devolver el dato al servidor, se utiliza para almacenar preferencias estéticas (como el modo oscuro), rastrear la procedencia de campañas de marketing y mantener identificadores anónimos que permiten personalizar la experiencia del usuario recurrente sin necesidad de una sesión activa en el servidor.

46. Base de datos

- **PDO :** Define una interfaz ligera y consistente para acceder a bases de datos, a diferencia de las extensiones específicas (como `mysqli`), PDO actúa como una capa de abstracción que permite al desarrollador interactuar con diferentes sistemas de gestión de bases de datos (MySQL, PostgreSQL, SQLite, SQL Server, etc.) utilizando el mismo conjunto de funciones y métodos. Internamente, el motor de PHP utiliza controladores específicos para cada motor de base de datos, pero expone una API orientada a objetos uniforme.

Se utiliza para estandarizar el acceso a datos en aplicaciones profesionales, facilitando la migración entre diferentes motores de bases de datos y proporcionando una base sólida para la seguridad y el manejo de transacciones.

|
| — **Conexión a base de datos :** Realiza instanciando un nuevo objeto de la clase
| PDO, este proceso requiere un DSN (Data Source Name), que es una cadena
| de texto que contiene el prefijo del controlador (ej. `mysql:`), el host, el nombre
| de la base de datos y, opcionalmente, el puerto y el juego de caracteres
| (charset), además se proporcionan las credenciales de acceso (usuario y
| contraseña) y un array opcional de atributos de configuración para definir el

comportamiento de la conexión (como el modo de error o el uso de emulación de sentencias).

Se utiliza para establecer el canal de comunicación persistente o volátil entre el script PHP y el servidor de base de datos, siendo el punto de partida obligatorio para cualquier operación de consulta o escritura.

— **prepared statements** : Característica de seguridad y rendimiento que separa la lógica de la consulta SQL de los datos que se le pasan, en lugar de concatenar variables directamente en la cadena SQL, el desarrollador envía una plantilla de consulta al servidor de base de datos con marcadores de posición (ya sean signos de interrogación ? o nombres con dos puntos :nombre) y el motor de la base de datos las compila y optimiza estas plantillas una sola vez.

Se utilizan como la defensa técnica definitiva contra ataques de Inyección SQL, ya que los datos de entrada se tratan estrictamente como parámetros y nunca como parte del comando ejecutable, garantizando que caracteres maliciosos no alteren la estructura de la consulta original.

— **bind parameters** : Asocia una variable de PHP a un marcador de posición específico dentro de una sentencia preparada, PDO ofrece métodos como `bindParam()` y `bindValue()` para realizar esta tarea, el primero vincula la variable por referencia (el valor se evalúa en el momento de la ejecución), mientras que el segundo lo hace por valor, durante este proceso, se puede especificar el tipo de dato esperado (como `PDO::PARAM_INT` o `PDO::PARAM_STR`), permitiendo que el controlador realice una validación y escape de datos adecuado antes de enviar la información al motor de base de datos.

Se utiliza para inyectar valores dinámicos de forma segura en consultas de inserción, actualización o filtrado, manteniendo una separación clara entre la estructura lógica y el contenido variable.

— **fetch modes** : Configuraciones de PDO que determinan el formato en el que se devuelven los resultados de una consulta tras su ejecución, mediante el método `setFetchMode()` o pasando una constante al método `fetch()`, el desarrollador puede elegir si desea recibir cada fila como un array asociativo (`PDO::FETCH_ASSOC`), un array indexado numéricamente (`PDO::FETCH_NUM`), un objeto genérico (`PDO::FETCH_OBJ`) o incluso mapear los datos directamente a una instancia de una clase personalizada (`PDO::FETCH_CLASS`).

Se utilizan para adaptar la salida de la base de datos a las necesidades específicas de la aplicación, optimizando el consumo de memoria y facilitando la integración de los datos con la lógica de objetos o plantillas de visualización.

— **Manejo de errores** : Configurados a través del atributo `PDO::ATTR_ERRMODE`, que define cómo debe reaccionar la extensión ante fallos en la conexión o en las consultas, existen tres modos principales: silencioso (por defecto), advertencia (`ERRMODE_WARNING`) y el estándar moderno de excepciones (`ERRMODE_EXCEPTION`), al activar el modo de

excepciones, cualquier fallo en la comunicación o sintaxis SQL disparará un objeto de la clase PDOException, el cual puede ser capturado mediante bloques try/catch.

Se utiliza para centralizar la detección de errores de base de datos, permitiendo realizar registros de log detallados, ejecutar reversiones de transacciones (rollback) y mostrar mensajes de error amigables sin exponer detalles técnicos internos de la infraestructura de datos.

- **Seguridad**

- └─ **Prevención de SQL injection** : Conjunto de técnicas de seguridad diseñadas para evitar que un atacante interfiera en las consultas que una aplicación realiza a su base de datos, este tipo de vulnerabilidad ocurre cuando datos de entrada no confiables se concatenan directamente en la cadena de comandos SQL, permitiendo que caracteres especiales (como comillas o puntos y coma) alteren la lógica de la consulta original para saltar autenticaciones, extraer información sensible o destruir tablas.

La solución técnica estándar en PHP es el uso de Sentencias Preparadas con Parámetros Vinculados a través de la extensión PDO, este método envía la estructura de la consulta al servidor de base de datos de forma separada a los datos, garantizando que el motor de la base de datos trate los valores externos estrictamente como texto o números y nunca como código ejecutable y se utiliza como un requisito obligatorio en cualquier punto del sistema donde se reciban datos de usuarios, APIs o archivos para interactuar con el motor de persistencia, blindando la infraestructura contra manipulaciones maliciosas del lenguaje de consulta.

- **Transacciones** : Mecanismo de control en el manejo de bases de datos que permite agrupar múltiples operaciones de escritura (INSERT, UPDATE, DELETE) en una única unidad lógica de trabajo, bajo el principio ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad), una transacción garantiza que, o bien todas las operaciones se ejecutan con éxito, o bien ninguna de ellas se aplica, dejando la base de datos en su estado original si ocurre un fallo en cualquier paso intermedio el motor de PHP inicia este proceso desactivando el modo de "autocommit" mediante el método beginTransaction().

Se utilizan en procesos críticos donde la integridad de la información depende de la correlación de varias tablas, como en transferencias bancarias, registros de pedidos con actualización de inventario o la creación simultánea de un usuario y sus perfiles de acceso.

- **Confirmación de Cambios** : mediante el método commit() se finaliza una transacción exitosa, indicando al servidor de base de datos que todas las operaciones pendientes dentro del bloque transaccional deben hacerse permanentes, es decir al invocar esta función, el motor de la base de datos consolida los cambios en el almacenamiento físico y los hace visibles para el resto de los usuarios y procesos del sistema y una vez ejecutado el commit, los cambios ya no pueden ser revertidos mediante las herramientas de control de transacciones de PHP.

Se utiliza al final de un bloque try tras verificar que todas las consultas necesarias se completaron sin errores, asegurando que la base de datos pase de un estado consistente a otro estado igualmente válido y actualizado.

- **Reversión de Estados** : Cancela una transacción en curso y deshace todos los cambios realizados desde que se invocó el inicio de la misma, al ejecutarse, la base de datos ignora todas las operaciones de escritura pendientes y restaura los datos a su estado exacto anterior al comienzo de la transacción.

Esta función se invoca típicamente dentro de un bloque catch tras capturar una excepción o detectar un error lógico durante el procesamiento de los datos y se utiliza para prevenir la "corrupción lógica" de la información (como restar dinero de una cuenta sin haberlo sumado en otra debido a un error de red o de sistema), garantizando que el sistema nunca mantenga datos parciales o inconsistentes ante un fallo inesperado.

47. Subida de archivos

- **\$_FILES** : Contiene toda la información de los elementos subidos al script mediante el método HTTP POST utilizando el tipo de codificación multipart/form-data, cada entrada en este array es a su vez un array asociativo que proporciona cinco datos críticos: name (nombre original del archivo), type (tipo MIME), tmp_name (ruta temporal donde el servidor almacenó el archivo), error (código de estado de la subida) y size (tamaño en bytes), el motor de PHP genera estos datos automáticamente al recibir la petición, manteniendo el archivo en una ubicación temporal restringida hasta que el script decida procesarlo o descartarlo.
Se utiliza como la interfaz de acceso única para capturar imágenes, documentos o cualquier recurso binario enviado por el usuario, permitiendo su inspección previa antes de ser movido al almacenamiento definitivo.
- **Validación de archivos** : Conjunto de comprobaciones lógicas y de seguridad que se ejecutan sobre los datos de \$_FILES antes de aceptar un recurso en el sistema de archivos del servidor, este proceso incluye la verificación del código de error (debe ser 0 o la constante UPLOAD_ERR_OK), la restricción del tamaño máximo permitido para evitar ataques de denegación de servicio (DoS) y, fundamentalmente, la validación del tipo de contenido, para una validación segura, no se debe confiar en la extensión del nombre o en el índice type enviado por el navegador, sino inspeccionar el contenido real mediante funciones como finfo_file() para determinar el tipo MIME verdadero.
Se utiliza para prevenir la subida de scripts maliciosos (como archivos .php camuflados), asegurar que las imágenes cumplan con dimensiones específicas y garantizar que el servidor solo almacene archivos que la aplicación sea capaz de procesar correctamente.
- **Persistencia de Recursos** : Mecanismo de seguridad estándar para transferir un archivo desde su ubicación temporal hacia un destino permanente en el servidor, realizando una comprobación interna crítica: verifica que el archivo especificado sea realmente un archivo subido mediante una petición HTTP POST legítima, impidiendo que un atacante intente manipular el script para mover archivos sensibles del

sistema operativo, si la validación es exitosa y el destino tiene los permisos de escritura adecuados, el archivo se traslada y la función devuelve true. Se utiliza como el paso final y obligatorio en cualquier flujo de subida de archivos, asegurando que el recurso sea almacenado de forma persistente y segura fuera de la carpeta temporal que el motor de PHP limpia automáticamente al finalizar la ejecución del script.

48. HTTP en PHP

- **Lectura del request** : Proceso por el cual el motor de PHP interpreta la petición entrante del cliente y desglosa sus componentes en estructuras de datos accesibles para el desarrollador, al recibir una solicitud, PHP analiza la línea de estado, las cabeceras (headers) y el cuerpo (body) del mensaje HTTP, los datos de la URL se inyectan automáticamente en la superglobal `_GET`, mientras que los datos del cuerpo se organizan en `_POST` o se exponen de forma cruda a través del flujo `php://input` para formatos como JSON o XML.
Se utiliza para capturar la intención del usuario, recuperar parámetros de búsqueda, procesar credenciales de acceso y recibir payloads de datos complejos en arquitecturas de servicios web y APIs.
- **Métodos HTTP** : Definen la acción semántica que el cliente desea realizar sobre un recurso del servidor, PHP identifica el método utilizado en la petición actual a través del índice `REQUEST_METHOD` de la superglobal `_SERVER`, los métodos más comunes en el desarrollo con PHP son GET (recuperación de datos), POST (envío y creación de recursos), PUT/PATCH (actualización de información) y DELETE (eliminación).
Se utilizan para implementar arquitecturas de tipo REST, permitiendo que un mismo punto de acceso (URL) ejecute diferentes lógicas de programación basándose exclusivamente en el verbo HTTP empleado por el cliente.
- **Headers** : Pares de clave-valor que viajan en la petición del cliente y en la respuesta del servidor para proporcionar información contextual sobre la transacción, en PHP, las cabeceras de entrada se recuperan mediante la función `getallheaders()` o a través de la superglobal `_SERVER` (prefijadas con `HTTP_`) y por otro lado, las cabeceras de salida se envían al cliente utilizando la función `header()`.
Se utilizan para definir el tipo de contenido (Content-Type), gestionar políticas de seguridad (CORS), controlar el almacenamiento en caché del navegador, enviar cookies de sesión y autenticar peticiones mediante tokens en las cabeceras de autorización.
- **Redirecciones** : Instrucción enviada por PHP al navegador del cliente indicándole que el recurso solicitado se encuentra en una ubicación diferente, esto se logra enviando una cabecera de respuesta Location junto con un código de estado HTTP adecuado (301 para permanente, 302 para temporal), al recibir esta cabecera el navegador interrumpe la carga actual y realiza una nueva petición de forma automática hacia la dirección especificada.
Se utilizan habitualmente para redirigir al usuario hacia un panel de control tras un inicio de sesión exitoso, enviar a los visitantes a versiones actualizadas de una

página o asegurar que los formularios no se reenvíen al recargar el navegador (patrón Post-Redirect-Get).

49. Seguridad básica

- **Contraseñas**

— **password_hash()** : Mecanismo oficial de PHP para crear una huella digital (hash) de una contraseña de forma segura e irreversible, a diferencia de funciones de hash genéricas, esta instrucción utiliza algoritmos robustos y actualizados (como Bcrypt o Argon2) que están diseñados específicamente para resistir ataques de fuerza bruta, el motor de PHP genera automáticamente un salt (sal única) criptográficamente seguro para cada contraseña y lo incluye dentro de la cadena de texto resultante, eliminando la necesidad de gestionar sales manualmente.

Se utiliza para transformar las contraseñas de texto plano de los usuarios en representaciones seguras antes de almacenarlas en la base de datos, garantizando que, incluso en caso de una filtración de datos, las credenciales originales permanezcan protegidas.

— **password_verify()** : Comprueba si una contraseña proporcionada por el usuario en texto plano coincide con un hash almacenado previamente en la base de datos, esta función es capaz de extraer internamente el algoritmo y el salt utilizados durante la creación del hash original para realizar una comparación precisa y segura. Una característica técnica crítica de esta función es que es resistente a ataques de tiempo (timing attacks), lo que significa que el tiempo que tarda en validar la contraseña es constante y no revela información sobre cuántos caracteres de la contraseña son correctos, se utiliza sistemáticamente en los procesos de inicio de sesión (login) para autenticar a los usuarios, permitiendo verificar la validez de una credencial sin necesidad de conocer o "desencriptar" la contraseña original.

- **Ataques comunes**

— **XSS** : Ocurrido cuando una aplicación web incluye datos no confiables en una página sin la validación o el escape adecuado, permitiendo a un atacante ejecutar scripts maliciosos (generalmente JavaScript) en el navegador del usuario final, en PHP esta vulnerabilidad se manifiesta típicamente al imprimir directamente en el HTML variables provenientes de `_GET`, `_POST` o bases de datos, existen dos variantes principales: el XSS reflejado (el script viaja en la URL) y el almacenado (el script se guarda en el servidor).

Se utiliza la función `htmlspecialchars()` para neutralizar esta amenaza, convirtiendo caracteres reservados como `<` y `>` en entidades HTML, asegurando que el navegador renderice el contenido como texto plano y no como código ejecutable.

— **CSRF** : Induce a un usuario autenticado a ejecutar acciones no deseadas en una aplicación web en la que confía, aprovechando que el navegador envía automáticamente las cookies de sesión en cada petición, un atacante puede

camuflar una solicitud maliciosa (como un cambio de contraseña o una transferencia de fondos) dentro de un enlace o una imagen en un sitio externo, para mitigar este riesgo en PHP, se implementan tokens anti-CSRF: cadenas aleatorias únicas generadas por el servidor, almacenadas en `_SESSION` e incluidas en campos ocultos de los formularios y al recibir la petición, el servidor valida que el token enviado coincida con el de la sesión, si no hay coincidencia, la petición se rechaza. Se utiliza para garantizar la integridad de las transacciones y asegurar que cada solicitud de cambio de estado sea legítima y originada por el usuario de forma consciente.

- **Seguridad de sesión**

└─ **`session_regenerate_id()`** : Reemplaza el identificador de la sesión actual por uno nuevo, manteniendo toda la información almacenada en el array `_SESSION` intacta, al ejecutarse, el motor de PHP genera un nuevo token único (PHPSESSID) y actualiza la cookie en el navegador del cliente, invalidando opcionalmente el identificador antiguo en el almacenamiento del servidor, esta operación es la defensa técnica principal contra el ataque de Fijación de Sesión (Session Fixation), donde un atacante intenta forzar a un usuario a utilizar un ID de sesión conocido para suplantar su identidad tras el login.
Se utiliza obligatoriamente en puntos críticos de cambio de privilegios, como al autenticar un usuario, cambiar contraseñas o realizar transacciones financieras, garantizando que el "hilo" de confianza entre el cliente y el servidor se renueve con un identificador fresco y seguro.

50. Consumo de APIs

- **`file_get_contents` con contexto** : Realiza peticiones HTTP complejas (como POST o PUT) mediante el uso de un stream context creado con `stream_context_create()`, el motor de PHP permite definir un array de opciones que incluye el método HTTP, las cabeceras personalizadas (como tokens de autorización) y el cuerpo de la petición (content), a diferencia de su uso simple para archivos locales, esta configuración transforma la función en un cliente HTTP ligero capaz de interactuar con APIs REST. Se utiliza para integraciones rápidas y sencillas donde no se requiere una gestión avanzada de la conexión, permitiendo recuperar el cuerpo de una respuesta externa directamente en una cadena de texto para su posterior decodificación.

- **`cURL` básico** : Extensión de PHP que proporciona una interfaz de programación mucho más potente y versátil para comunicarse con servidores externos a través de diversos protocolos (HTTP, HTTPS, FTP, etc.), El flujo de trabajo básico consiste en cuatro etapas: inicializar la sesión con `curl_init()`, configurar las opciones de transferencia con `curl_setopt()` (como la URL, el método y el retorno de la transferencia), ejecutar la petición con `curl_exec()` y cerrar el recurso con `curl_close()`.

A diferencia de los flujos de archivos, `cURL` ofrece un control granular sobre los tiempos de espera (timeouts), el manejo de cookies, la verificación de certificados SSL y la recuperación detallada de metadatos de la red, se utiliza como el estándar

de la industria para el desarrollo de clientes de API profesionales y la integración de servicios de terceros complejos.

- **status codes** : Respuestas numéricas enviadas por el servidor de la API para indicar el resultado de una petición, PHP permite acceder a estos códigos tras una ejecución de cURL mediante `curl_getinfo($ch, CURLINFO_HTTP_CODE)` o a través de la variable `$http_response_header` cuando se usan streams, los códigos se dividen en categorías: 2xx para éxito (como el 200 OK o 201 Created), 4xx para errores del cliente (como 400 Bad Request o 401 Unauthorized) y 5xx para errores del servidor.

Se utilizan para implementar la lógica de control de flujo en el consumo de APIs, determinando si los datos recibidos son válidos para ser procesados o si se debe ejecutar una rutina de reintento o gestión de fallos.

- **Manejo de errores** : Consiste en la detección y gestión de fallos que ocurren durante la comunicación de red o en la validación de la respuesta del servidor externo, en cURL esto se logra mediante funciones como `curl_errno()`, que devuelve un código de error de red (como problemas de DNS o conexión rechazada), y `curl_error()`, que proporciona una descripción textual del incidente, además un manejo robusto implica validar que el contenido recibido sea una estructura de datos válida (usando `json_last_error()`) antes de intentar manipularla.

Se utiliza para prevenir que una caída en un servicio externo bloquee la ejecución del script local, permitiendo registrar logs de depuración y ofrecer respuestas de contingencia al usuario final.

51. CLI

- **¿Qué es?** : Opera directamente desde la terminal o consola del sistema operativo, independientemente de un servidor web (como Apache o Nginx), en este entorno el motor de PHP se invoca mediante el binario `php`, permitiendo la ejecución de archivos `.php` de forma aislada y a diferencia del modo web (SAPI), el CLI no genera cabeceras HTTP, tiene un tiempo de ejecución ilimitado por defecto (`max_execution_time = 0`) y dirige la salida de datos directamente a la interfaz estándar de la consola (STDOUT).

Se utiliza para la automatización de tareas administrativas, la creación de herramientas de desarrollo, la ejecución de pruebas unitarias y el procesamiento de procesos en segundo plano (background jobs) que requieren un consumo de recursos sostenido.

- **scripts de consola** : Programa escrito en PHP cuya lógica está optimizada para interactuar con el sistema de archivos, bases de datos o servicios de red desde la terminal, estos scripts suelen comenzar con una línea especial llamada Shebang (`#!/usr/bin/php`), que indica al sistema operativo qué intérprete debe utilizar para ejecutar el archivo, al no depender de una petición de navegador, estos scripts pueden ser programados mediante tareas cron (crontab) para ejecutarse a intervalos regulares.

Se utilizan para realizar migraciones de bases de datos, limpiezas periódicas de caché, generación de reportes masivos, envío de correos electrónicos en lotes y

cualquier operación de mantenimiento que deba ser independiente de la interacción del usuario final.

- **Argumentos de entrada** : Parámetros que se pasan a un script PHP al momento de su invocación desde la terminal (ej. `php script.php argumento1 argumento2`), el motor de PHP capturará automáticamente estos valores y los organiza en dos variables globales especiales: `$argc`, que almacena el número total de argumentos pasados (incluyendo el nombre del script), y `$argv`, que es un array indexado que contiene las cadenas de texto de cada argumento.
Se utilizan para dotar a los scripts de consola de dinamismo, permitiendo al desarrollador especificar nombres de archivos, identificadores de registros, flags de configuración o modos de ejecución sin necesidad de modificar el código fuente, facilitando la creación de herramientas de línea de comandos versátiles y potentes.

52. Autoloading

- **¿Que es?** : Permite cargar automáticamente el archivo de definición de una clase, interfaz o trait en el momento exacto en que se intenta utilizar por primera vez, en lugar de incluir manualmente cada archivo mediante sentencias `require` o `include` al inicio del script, el sistema de carga automática detecta cuándo se invoca una entidad no definida y busca su archivo correspondiente en el sistema de archivos. Se utiliza para mejorar el rendimiento de la aplicación al cargar únicamente los componentes necesarios para la petición actual (Lazy Loading), reducir la redundancia de código y mantener una estructura de proyecto organizada y escalable bajo estándares de interoperabilidad como PSR-4.
- **`spl_autoload_register()`** : Instrucción oficial de la biblioteca estándar de PHP (SPL) utilizada para registrar una o más funciones de carga personalizada en la "cola de autoloading", cuando el intérprete encuentra una clase que no ha sido declarada, recorre secuencialmente todas las funciones registradas con esta instrucción, pasándoles como argumento el nombre completo de la clase buscada (incluyendo su namespace) y el desarrollador define dentro de esta función la lógica para convertir ese nombre de clase en una ruta de archivo física (por ejemplo, transformando `App\Models\User` en `src/Models/User.php`).
Se utiliza para implementar sistemas de carga automática robustos y flexibles, permitiendo que múltiples librerías y frameworks registren sus propios cargadores sin entrar en conflicto entre sí.

53. Debugging

- **logs** : Registros cronológicos de eventos, errores o transacciones que ocurren durante la ejecución de un script., representando la memoria persistente del comportamiento del servidor y son la herramienta primaria para el diagnóstico de fallos en entornos de producción donde no es posible realizar una depuración interactiva, un log técnico debe contener, como mínimo, la fecha y hora del evento, el nivel de severidad (`info`, `warning`, `error`), un mensaje descriptivo y, en casos críticos, el archivo y línea de origen.
Se utilizan para monitorizar la salud del sistema, auditar acciones de usuarios, detectar intentos de intrusión y analizar el rendimiento de procesos en segundo plano que no tienen una salida visual directa.

- **error_log()** : Envía mensajes personalizados al sistema de registro de errores del servidor o a un destino específico definido por el desarrollador, esta función acepta cuatro parámetros principales: el mensaje a registrar, el tipo de registro (sistema, correo electrónico o archivo), el destino y las cabeceras adicionales, y al invocarse, el motor de PHP formatea el mensaje y lo envía al flujo correspondiente, asegurando que la información quede persistida incluso si el script termina de forma abrupta. Se utiliza para capturar excepciones en bloques catch, registrar variables de depuración sin interrumpir la salida HTML del usuario y documentar fallos en comunicaciones con servicios externos o bases de datos.
- **logging a archivo** : Práctica de dirigir los mensajes de error o depuración hacia un archivo de texto plano (generalmente con extensión .log) dentro del sistema de archivos del servidor, para implementar esto mediante error_log(), se utiliza el tipo de mensaje 3 y se especifica la ruta absoluta del archivo destino, es fundamental que el proceso que ejecuta PHP (como www-data) tenga permisos de escritura en el directorio seleccionado.
Esta técnica permite segmentar los registros por categorías (por ejemplo, un archivo para errores de pagos y otro para accesos) y se utiliza para mantener un historial organizado de la aplicación, facilitar la rotación de logs mediante herramientas del sistema operativo y permitir que administradores de sistemas analicen el comportamiento de la plataforma sin acceder directamente al código fuente.